

Ansible - Part 2

6-MONTH INDUSTRY MASTERCLASS PATHWAY

An enterprise-grade, comprehensive technical architecture, deployment workflow, security hardening guide, and observability plan. Designed for senior engineering professionals (6+ years experience) striving for Principal SRE and Cloud Architect roles.

CORE CONCEPTS | PRODUCTION HARDENING | 20+ INCIDENT SCENARIOS

Automated SRE Learn System | Generated Pdf Generator

Ansible Production Study Guide (Part 2 of 3)

1. Part Introduction & Scope

This guide covers the advanced architecture, performance tuning, security hardening, sandboxing, and scale boundaries of Ansible.

[Ansible Scale & Security Domain]		
?		
??		
?	?	?
[Performance]	[Security]	[Sandboxing]
??? Pipelining	??? Vault & KMS	??? Execution Envs
??? ControlPersist	??? Priv Escalation	??? Ansible-BUILDER
??? Mitogen Engine	??? No-Log Policies	??? Ansible-NAVIGATOR

Scope of this Guide

- **Advanced Configurations:** Deep dive into `ansible.cfg` parameters, fact caching engines (Redis/Memcached), and execution strategies (`linear`, `free`, `host\_pinned`).
- **Performance Tuning:** Optimizing network round-trips via SSH multiplexing, pipelining, and Mitogen; mitigating Control Node memory/CPU bottlenecks.
- **Security & Hardening:** Enterprise secret management (Ansible Vault, HashiCorp Vault lookup plugins), privilege escalation (`sudo`/`become` hardening), and execution sandboxing.
- **Sandboxing & Portability:** Designing, building, and running Ansible Execution Environments (EE) using `ansible-builder` and `ansible-navigator`.
- **Scale Boundaries:** Managing orchestration across \$5,000+\$ nodes using Ansible Automation Platform (AAP) Automation Mesh, receptor networks, and hop nodes.

2. Why These Concepts are Critical for High-Availability Systems

Minimizing MTTR (Mean Time to Resolution)

In high-availability (HA) systems, configuration drift or security vulnerabilities must be remediated across thousands of servers simultaneously. If a critical zero-day vulnerability requires a patch, a default Ansible configuration (which processes hosts slowly in small batches) can take hours.

By utilizing pipelining, optimized forks, and the `free` execution strategy, SREs can reduce execution times by up to 90%, dropping remediation windows from hours to minutes.

Preventing Blast Radius Propagation

Misconfigured execution strategies or unconstrained parallel runs can trigger cascading failures. For instance, running an unthrottled playbook across an entire database cluster can cause simultaneous restarts, leading to a complete service outage.

Understanding `serial` boundaries, `max_fail_percentage` controls, and execution strategies allows architects to design rolling updates that preserve quorum and maintain high availability.

Preventing Secret Leaks and Privilege Escalation Exploits

In multi-tenant cloud environments, Ansible controllers often possess high-privilege credentials. Without strict sandboxing, hardened SSH configurations, and automated secret redaction (`no_log`), a compromise of the Ansible Control Node can lead to complete lateral compromise of the entire cloud infrastructure.

3. Real-World Enterprise Use Cases

Use Case 1: Zero-Downtime Rolling Upgrades across 5,000+ Multi-Region Nodes

- **Objective:** Deploy a security patch to a globally distributed web application cluster across 3 regions (AWS, GCP, on-premise) without dropping a single active HTTP request.
- **Challenge:** High latency across regions (e.g., US-East to EU-Central) makes standard SSH connections slow. A single failing node must not halt the entire global deployment, but if more than 10% of nodes in a single region fail, the deployment must immediately roll back.
- **Architectural Solution:**
- **Execution Strategy:** Use the `linear` strategy combined with dynamic `serial` blocks (e.g., `[10%, 20%, 50%]`) to limit the initial blast radius.
- **Transport Optimization:** Enable SSH `ControlPersist` and `pipelining` to reuse SSH connections, mitigating cross-region latency.
- **Failure Thresholds:** Set `max_fail_percentage: 10` to halt execution if a regional cluster shows systemic failure.

```
[Control Node]
?
??????? (Serial: 10%) ?????? [Canary Nodes (10%)] ??? (Success?)
?                                                    ? (Yes)
??????? (Serial: 20%) ?????? [Batch 2 Nodes (20%)] ????????
?                                                    ? (Yes)
??????? (Serial: 70%) ?????? [Remaining Nodes (70%)] ???????
```

Use Case 2: Multi-Tenant Security Hardened Patching Engine

- **Objective:** Provide a self-service OS patching pipeline for 50+ independent engineering teams sharing a single Kubernetes-based Ansible execution platform.
- **Challenge:** Prevent Team A from accessing Team B's SSH keys or inventories, and ensure that custom execution dependencies (e.g., specific Python libraries or CLI binaries) do not conflict.
- **Architectural Solution:**
- **Sandboxing:** Package Ansible, collection dependencies, and system libraries into custom **Execution Environments (EE)** using `ansible-builder`.`
- **Isolation:** Run EEs as ephemeral, unprivileged containers inside a Kubernetes cluster, managed by Ansible Runner.
- **Secrets Management:** Integrate with HashiCorp Vault via AppRole authentication, dynamically injecting temporary SSH keys and target credentials into memory at runtime, leaving no secrets on disk.

4. Comprehensive Architecture Explanation

The Ansible execution engine manages the transition from declarative YAML playbooks to imperative commands executed on remote nodes. Below is the internal execution architecture of a highly optimized, sandboxed Ansible run:

```
graph TB
    subgraph Control_Node_Host [Control Node Host]
        subgraph EE_Container [Execution Environment Container]
            direction TB
            AV[Ansible Engine / Playbook] -->|Decrypts Secrets| AVault[Ansible Vault Engine]
            AV -->|Generates Task JSON| AP[Ansible Plugins & Modules]
            AP -->|Executes| AR[Ansible Runner]
        end

        AR -->|Spawns Multiplexed Connections| SSH[SSH ControlMaster Daemon]
        HV[HashiCorp Vault] -->|Dynamic Secrets| AVault
    end

    subgraph Network_Boundary [Network Security Zone]
        SSH -->|Port 22 / ProxyJump| Bastion[SSH Bastion / Proxy]
    end

    subgraph Target_Nodes [Target Infrastructure]
        Bastion -->|Pipelined Python Payload| Target1[Target Node 1 /tmp]
        Bastion -->|Pipelined Python Payload| Target2[Target Node 2 /tmp]
    end

    classDef container fill:#e1f5fe,stroke:#01579b,stroke-width:2px;
    classDef host fill:#fff3e0,stroke:#e65100,stroke-width:2px;
```

```
classDef target fill:#e8f5e9,stroke:#1b5e20,stroke-width:2px;
class EE_Container container;
class Control_Node_Host host;
class Target_Nodes target;
```

Architectural Component Deep Dive

1. Ansible Runner & Execution Environments (EE)

Instead of running directly on the host OS, Ansible executes inside a containerized Execution Environment. This isolates the execution engine, guaranteeing that Python dependencies, Ansible collections, and system binaries are identical across local development, CI/CD pipelines, and production environments.

2. Secret Decryption & Lookup Engines

During the compilation phase, Ansible decrypts Vault-encrypted variables in memory. If integrated with an external KMS or HashiCorp Vault, dynamic lookups are performed over TLS, fetching transient credentials that exist only within the ephemeral memory space of the runner container.

3. SSH Multiplexing (ControlMaster/ControlPersist)

By default, SSH establishes a new TCP handshake and authentication session for every single task. With `ControlMaster` enabled, the Control Node establishes a persistent master connection (`ControlPersist`) on the first task. Subsequent tasks send payload data over this pre-existing socket, reducing network overhead.

4. Pipelining

Normally, Ansible copies a module payload to the remote host's temporary directory via SFTP/SCP, executes it via SSH, and deletes the temporary file.

When pipelining is enabled, Ansible executes the remote Python interpreter directly over the SSH connection and pipes the module code straight into `stdin`. This bypasses disk writes on the target node and eliminates multiple SSH round-trips per task.

5. Types, Classifications, and Components

Execution Strategies

Strategy	Execution Pattern	Best Used For	Drawbacks
linear	Default	Executes one task across all hosts in the current batch (defined by `serial`) before moving	

| :--- | :--- | :--- | :--- |

| `linear` *(Default)* | Executes one task across all hosts in the current batch (defined by `serial`) before moving

to the next task. | Highly synchronized deployments (e.g., database migrations that must complete before application servers update). | Slowest overall execution; fast hosts are blocked waiting for slow hosts to complete the task. |

| ``free`` | Each host runs the entire playbook as fast as possible, independent of other hosts. | Mass patching, security auditing, and independent agent installations. | Cannot coordinate tasks across hosts (e.g., cannot guarantee a load balancer is updated only after web servers are ready). |

| ``host_pinned`` | Each host runs all tasks in sequence, but execution is constrained by the ``forks`` limit. | Running complex multi-task playbooks across large inventories where individual host completion is preferred over step-by-step synchronization. | High resource consumption on the control node if ``forks`` is set too high. |

Connection Plugins

[Connection Plugins]		
?	?	?
??		
?	?	?
[Standard SSH]	[Mitogen]	[Local]
- OpenSSH binary	- Custom Python protocol	- Direct process execution
- Native multiplexing	- Bypasses Ansible loop	- Used for localhost
- Broad compatibility	- Drastically faster	- High performance

- `**ssh`` (Default):\*\* Uses the system's OpenSSH binary. Supports all SSH features including ``ProxyJump``, SSH keys, and hardware tokens.
- `**paramiko``:** A pure Python implementation of SSHv2. Used as a fallback when OpenSSH is unavailable or on legacy platforms. It does not support native SSH multiplexing.
- `**mitogen`` (Third-Party):** A highly optimized connection plugin that replaces the standard Ansible execution loop. It uses a custom pipelining protocol that runs tasks much faster, but it is not compatible with all Ansible features (e.g., some complex privilege escalations).
- `**local``:** Runs commands directly on the control host, bypassing the network stack entirely.

Privilege Escalation Methods

- `**sudo``:\*\* The industry standard. Executes commands as another user (typically ``root``) using the system ``sudo`` configuration.
- `**su``:\*\* Switches users using the ``su`` binary. Requires managing the target user's password directly.
- `**pbrun`` / ``pfexec`` / ``doas``:** Enterprise alternatives used on hardened Unix systems or Solaris to execute privileged actions under strict RBAC constraints.

6. Step-by-Step Production Implementation Guide

This guide details how to build, configure, and execute a highly optimized, sandboxed, and secure Ansible

automation run.

Step 1: Define the Execution Environment Specification

Create an `execution-environment.yml` file to define the container image containing all required dependencies.

```
---
version: 3

images:
  base_image:
    name: registry.redhat.io/ansible-automation-platform-24/ee-minimal-rhel8:latest

dependencies:
  galaxy:
    collections:
      - amazon.aws
      - community.general
      - community.crypto
  python:
    - hvac>=2.1.0 # Required for HashiCorp Vault integration
    - netaddr>=0.10.1 # IP address manipulation
    - cryptography>=42.0.0
  system:
    - openssh-clients
    - rsync
    - tar

options:
  package_manager_path: /usr/bin/microdnf
```

Step 2: Build the Execution Environment Image

Run `ansible-builder` to compile this definition into a production-ready container image.

```
# Build the container image locally
ansible-builder build \
  --tag enterprise-ansible-ee:v1.0.0 \
  --container-runtime docker \
  --verbosity 3
```

Step 3: Configure the Hardened SSH Multiplexing Environment

On the Control Node (or inside the Execution Environment), create a secure SSH configuration file (`~/.ssh/config`) to manage persistent connections.

```
# Secure & Optimized SSH Configuration for Ansible Control Node
```

```
Host *
  # Enable connection multiplexing
  ControlMaster auto
  # Path to store the multiplexing socket
  ControlPath ~/.ssh/ansible-%r@%h:%p
  # Keep the connection open for 4 hours after inactivity
  ControlPersist 4h
  # Security: Disable unsafe authentication methods
  PasswordAuthentication no
  PubkeyAuthentication yes
  # Mitigate SSH connection timeouts over slow WAN links
  ServerAliveInterval 30
  ServerAliveCountMax 3
  # Strict Host Key Checking
  StrictHostKeyChecking yes
  UserKnownHostsFile ~/.ssh/known_hosts
```

7. Standard CLI Commands with Deep Technical Explanations

1. `ansible-playbook`

Executes an Ansible playbook.

```
ansible-playbook -i inventories/prod/site.ini playbooks/deploy.yml \
--forks 50 \
--connection ssh \
--inventory-directory inventories/prod/ \
--limit "webserver:&region_us_east" \
--vault-id prod@.vault_pass \
--check \
--diff
```

- `--forks 50`: Spawns up to 50 parallel worker processes. This allows Ansible to configure up to 50 remote hosts simultaneously.
- `--limit "webserver:®ion_us_east"`: Restricts execution to hosts that belong to **both** the `webserver` group and the `region_us_east` group. This is a safe way to limit the blast radius of a playbook run.
- `--vault-id prod@.vault_pass`: Specifies the vault password file to decrypt variables tagged with the label `prod`. This avoids prompt-based automation blocks.
- `--check`: Runs the playbook in dry-run mode. Modules predict what changes would occur but do not modify the remote system.
- `--diff`: Displays the exact line-by-line changes (in unified diff format) that would be made to files on the remote hosts.

2. `ansible-vault`

Manages encrypted variables and files.

```
ansible-vault encrypt_string 'super-secret-password' \  
--name 'db_root_password' \  
--vault-id prod@.vault_pass
```

- `encrypt\_string`: Encrypts a single string instead of an entire file, allowing you to safely commit variables to version control.
- `--name 'db\_root\_password'`: Automatically formats the output as a YAML variable named `db\_root\_password`.
- `--vault-id prod@.vault\_pass`: Uses the key inside `.vault\_pass` labeled `prod` to perform AES-256 encryption.

3. `ansible-navigator`

A container-native CLI tool used to run and debug playbooks inside Execution Environments.

```
ansible-navigator run playbooks/deploy.yml \  
--eei enterprise-ansible-ee:v1.0.0 \  
--execution-environment true \  
--mode stdout \  
--ll debug
```

- `--eei enterprise-ansible-ee:v1.0.0`: Forces execution to run inside the specified custom Execution Environment container image.
- `--execution-environment true`: Enables containerized execution sandboxing.
- `--mode stdout`: Forces standard output mode, which is ideal for CI/CD pipelines where interactive TUI (Text User Interface) screens cannot be rendered.

8. Production Configuration Examples

Hardened & Optimized `ansible.cfg`

This configuration file is optimized for high-throughput, low-latency execution across thousands of nodes while enforcing strict security controls.

```
[defaults]  
# --- Performance Optimization ---  
# Increase parallel worker processes (Forks)  
forks = 50  
# Gather facts only when explicitly requested, or cache them  
gathering = smart
```

```
# Use Redis for fact caching to avoid gathering facts repeatedly
fact_caching = redis
fact_caching_connection = localhost:6379:0
fact_caching_timeout = 86400
# Enable custom callbacks to output execution times
callbacks_enabled = ansible.posix.profile_tasks, community.general.opentelemetry

# --- Security Hardening ---
# Block execution if host key is unknown (prevents MitM attacks)
host_key_checking = True
# Disable command execution logging to prevent secret leaks in logs
no_target_syslog = True
# Configure strict permissions for temporary files
remote_tmp = ~/.ansible/tmp
local_tmp = ~/.ansible/tmp

# --- Output Formatting ---
stdout_callback = yaml
bin_ansible_callbacks = True

[ssh_connection]
# --- Network Optimization ---
# Enable pipelining to execute commands directly in memory over SSH
pipelining = True
# Re-use active SSH connections to skip authentication overhead
ssh_args = -o ControlMaster=auto -o ControlPersist=30m -o ConnectionAttempts=100
# Path format for the multiplexing socket (must be short to avoid path length limits)
control_path = %(directory)s/ansible-ssh-%%h-%%p-%%r

[privilege_escalation]
# --- Security Hardening ---
become = True
become_method = sudo
become_user = root
become_ask_pass = False
```

Hardened Production Playbook

This playbook demonstrates how to run rolling updates securely using variable masking, vault integration, and failure thresholds.

```
---
- name: Enterprise Zero-Downtime Rolling Web Update
  hosts: webservers
  become: true
  # Execute in batches to maintain service availability
  serial:
    - 1
```

```
- 10%
- 50%
# Abort the entire run if more than 5% of nodes in a batch fail
max_fail_percentage: 5

vars_files:
  - vars/vaulted_secrets.yml

tasks:
  - name: Ensure internal application config is deployed
    ansible.builtin.template:
      src: templates/app_config.j2
      dest: /etc/app/config.json
      owner: root
      group: root
      mode: '0600' # Strict read/write permissions
      register: config_update

  - name: Securely write database password to local config
    ansible.builtin.lineinfile:
      path: /etc/app/credentials
      line: "DB_PASS={{ vault_db_password }}"
      owner: root
      group: root
      mode: '0600'
    # Prevent sensitive variable values from being written to logs/stdout
    no_log: true

  - name: Gracefully restart application service
    ansible.builtin.systemd:
      name: webapp
      state: restarted
    when: config_update.changed
    register: service_restart

  - name: Verify application health locally
    ansible.builtin.uri:
      url: http://localhost:8080/health
      status_code: 200
    # Wait up to 30 seconds for the application to start
    register: health_check
    until: health_check.status == 200
    retries: 6
    delay: 5
```

9. Security Considerations & Hardening Best Practices

```
[Ansible Hardening Framework]
?
????????????????????????????????????????????????????????
?                                ?                                ?
[Secret Shielding]      [Access Control]      [Network Isolation]
??? HashiCorp Vault     ??? Least Privilege   ??? Bastion Hosts
??? no_log: true        ??? Strict Sudoers    ??? ProxyJump Config
```

Secret Shielding

- **Externalize Secrets:** Avoid committing Ansible Vault files directly to git if possible. Instead, use lookup plugins to fetch secrets directly from enterprise KMS providers like HashiCorp Vault or AWS Secrets Manager at runtime:

```
db_password: "{{ lookup('community.hashivault.vault_kv2_get',
'secret/data/db').secret.password }}"
```

- **Strict Task Masking:** Always apply `no_log: true` to tasks that handle sensitive data, such as database passwords, API keys, or private certificates. This prevents Ansible from printing the values to standard output or writing them to system logs.

Access Control & Least Privilege

- **Limit Sudo Scope:** Do not configure `become: true` globally across an entire playbook unless absolutely necessary. Instead, apply it only to specific tasks that require elevated privileges.
- **Restrict Sudoers Configuration:** On target nodes, restrict the Ansible execution user's `sudo` privileges to only the specific commands required for maintenance. Avoid configuring a broad `ALL=(ALL) NOPASSWD: ALL` in the sudoers file.

Network Isolation & Zoning

- **Use Bastion Hosts (Jump Boxes):** Never expose management ports (like SSH port 22) directly to the public internet. Instead, route all Ansible traffic through a hardened bastion host using SSH `ProxyJump`:

```
ansible_ssh_common_args: '-o ProxyJump=bastion.enterprise.internal'
```

- **Enforce Host Key Verification:** Always keep `host_key_checking = True` enabled in production. Disabling this setting exposes your management traffic to Man-in-the-Middle (MitM) attacks.

10. Observability & Monitoring Considerations

Prometheus Metrics to Monitor (AWX/AAP Controller)

Metric Name	Type	Description	Alerting Threshold
---	---	---	---
`awx_system_info_cpu_percentage`	Gauge	CPU usage on the automation controller node.	`> 85%` sustained for 5 minutes.
`awx_system_info_memory_percentage`	Gauge	Memory utilization of the execution engine.	`> 90%` (risk of OOM killer terminating runs).
`awx_jobs_running_total`	Gauge	Number of active concurrent playbooks.	High values indicate potential capacity exhaustion.
`awx_job_status_failed_total`	Counter	Cumulative count of failed automation jobs.	Alert if rate of increase exceeds `5%` of total runs.

Log Aggregation & Analysis

To monitor execution health across your infrastructure, configure Ansible to output structured JSON logs to a centralized log aggregator (such as an ELK stack or Splunk).

1. Enable Structured JSON Output

Install and configure the `ansible.posix.json` callback plugin in your `ansible.cfg`:

```
[defaults]
stdout_callback = community.general.json
```

2. Log Analytics Queries (Elasticsearch / Kibana KQL)

To identify tasks that are causing performance bottlenecks, search for executions where task runtimes exceed acceptable limits:

```
# Find tasks that took longer than 10 seconds to execute
ansible_task_duration_seconds : > 10
```

To monitor security events, look for authorization or privilege escalation failures:

```
# Detect privilege escalation failures on target nodes
message: "sudo: auth failure" OR message: "Permission denied"
```

11. Common Troubleshooting Scenarios with RCA Steps

Scenario 1: SSH Connection Timeouts / "Shared connection to ... closed"

- **Symptom:** Playbook runs fail randomly during task execution with an error like `unreachable: Shared connection to 10.0.4.12 closed`.

- **Root Cause Analysis (RCA):**

1. ControlPersist Socket Expiry: The persistent SSH socket might be closed by the target host's idle timeout configuration (`ClientAliveInterval`).
2. MTU Size Mismatch: Network packets containing larger Ansible payloads may be dropped by intermediate routers if the MTU size is misconfigured.

- **Resolution Steps:**

1. Test connection stability with pipelining disabled to isolate the issue:

```
ANSIBLE_SSH_PIPELINING=False ansible -i inventory -m ping all
```

2. If the ping succeeds without pipelining, adjust your SSH arguments in `ansible.cfg` to send keepalive packets:

```
ssh_args = -o ControlMaster=auto -o ControlPersist=15m -o ServerAliveInterval=10 -o
ServerAliveCountMax=3
```

Scenario 2: Privilege Escalation Failures ("Missing sudo password")

- **Symptom:** The playbook fails with `Missing sudo password` or `sudo: a password is required`.
- **Root Cause Analysis (RCA):**
 - The playbook is configured with `become: true`, but the SSH user does not have passwordless sudo privileges on the target host. Additionally, no escalation password was provided to the runner.
- **Resolution Steps:**

1. Verify the target host's `/etc/sudoers` file contains the correct configuration for the Ansible user:

```
ansible_user ALL=(ALL) NOPASSWD: ALL
```

2. If passwordless sudo is disabled by policy, run the playbook with the `--ask-become-pass` flag to securely provide the escalation password:

```
ansible-playbook -i inv site.yml --ask-become-pass
```

Scenario 3: Memory Exhaustion (OOM) on Control Node

- **Symptom:** The Ansible execution process is terminated abruptly with `Killed` or `Out of memory` logged in `/var/log/messages`.
- **Root Cause Analysis (RCA):**
 - The `forks` count is set too high relative to the Control Node's available RAM. Each worker process consumes memory to parse playbooks, compile tasks, and store gathered host facts in memory.
- **Resolution Steps:**

1. Calculate the safe maximum forks limit using this formula:

$$\text{Max Forks} = \frac{\text{Available RAM (MB)} - 2048 \text{ MB}}{\text{Average Memory Per Fork (approx. 80-120 MB)}}$$

2. If memory usage remains high, enable fact caching to an external store like Redis to offload memory usage from the main process:

```
fact_caching = redis
fact_caching_connection = localhost:6379:0
```

12. Common Mistakes and How to Avoid Them in Production

Mistake 1: Relying on Default `forks = 5` on Large Inventories

- **The Error:** Running playbooks with the default configuration across hundreds of nodes. This forces Ansible to process hosts in small batches of 5, which drastically slows down deployments.
- **The Fix:** Increase the `forks` value in `ansible.cfg` to match your control node's hardware capacity (e.g., `forks = 50` or `100`).

Mistake 2: Storing Plain-Text Secrets in Git Repositories

- **The Error:** Committing database passwords, private keys, or API tokens in plain text within your playbook variables.
- **The Fix:** Always encrypt sensitive variables using `ansible-vault encrypt\_string` or fetch them dynamically from an external secrets manager at runtime.

Mistake 3: Overusing Shell and Command Modules

- **The Error:** Writing playbooks that rely heavily on raw shell commands:

```
- name: Install Apache (Anti-Pattern)
  ansible.builtin.shell: apt-get install -y apache2
```

This approach bypasses Ansible's state management, making tasks non-idempotent and prone to failure on subsequent runs.

- **The Fix:** Use native, declarative modules that guarantee idempotency:

```
- name: Install Apache (Production Best Practice)
  ansible.builtin.apache2:
    name: apache2
    state: present
```

13. Enterprise-Level Recommendations

Performance Tuning Checklist

1. Enable Pipelining: Ensure `pipelining = True` is active in `ansible.cfg`. Note that you must disable `requiretty`

in the `/etc/sudoers` file on target nodes for this to work.

2. Configure `ControlPersist`: Set a long persistence window (e.g., `ControlPersist 30m`) so that SSH connections are kept open and reused across playbooks.

3. Use Fact Caching: Avoid gathering facts repeatedly by storing them in a shared Redis cache with a 24-hour expiration time.

Connection Pooling Architecture

```
[Ansible Engine]
  ?
  ?
[ControlMaster Multiplexing Daemon]
  ?
  ?? (Pre-established TCP Socket) ?? [Target Host A]
  ?? (Pre-established TCP Socket) ?? [Target Host B]
  ?? (Pre-established TCP Socket) ?? [Target Host C]
```

By maintaining persistent SSH connections, you eliminate the overhead of TCP handshakes and key exchanges for every task. This significantly reduces execution times, especially when managing remote nodes over high-latency networks.

14. Advanced Concepts

Ansible Automation Mesh

For large-scale, multi-region deployments, a single control node cannot easily manage connections across network boundaries or firewalls. Ansible Automation Mesh solves this by establishing a decentralized, peer-to-peer execution network using Receptor nodes.

```

      [Ansible Controller]
          ?
      (Control Node)
          ?
      [Hop Node (DMZ)]
          ?
      ?????????????????????????????????????????
      ?                                     ?
[Execution Node (AWS)]      [Execution Node (GCP)]
      ?                                     ?
      (Target Nodes)          (Target Nodes)
```

- ****Control Nodes:**** Run the main controller services and schedule jobs.

- **Hop Nodes:** Act as network relays. They route traffic across firewalls and DMZs, passing execution commands to isolated networks.
- **Execution Nodes:** Receive tasks from hop nodes and run them locally against target systems, reporting the results back through the mesh.

Custom Ansible Filter Plugins

You can extend Ansible's capabilities by writing custom Python filter plugins to process data within your playbooks.

Save this script as `filter_plugins/custom_filters.py`:

```
class FilterModule(object):
    def filters(self):
        return {
            'mask_ip_address': self.mask_ip_address
        }

    def mask_ip_address(self, ip_string):
        """
        Masks the last octet of an IP address for security logging.
        Example: 192.168.1.15 -> 192.168.1.***
        """
        try:
            parts = ip_string.split('.')
            if len(parts) == 4:
                return f"{parts[0]}.{parts[1]}.{parts[2]}.***"
            return ip_string
        except Exception:
            return ip_string
```

Usage in a Playbook:

```
- name: Log masked server IP
  ansible.builtin.debug:
    msg: "The target server IP is {{ ansible_default_ipv4.address | mask_ip_address }}"
```

15. Integration with Other DevOps Tools

HashiCorp Terraform Integration

Use Terraform to provision your cloud infrastructure, and then hand off configuration management to Ansible using dynamic inventories or local execution blocks.

```
# Terraform resource definition with Ansible hand-off
resource "aws_instance" "web" {
  ami          = "ami-0c55b159cbfaffe1f0"
  instance_type = "t3.medium"
  tags = {
    Name = "web-prod-01"
    Role = "webservers"
  }

  # Use a local-exec provisioner to trigger Ansible once the instance is ready
  provisioner "local-exec" {
    command = <<EOT
      aws ec2 wait instance-status-ok --instance-ids ${self.id} && \
      ansible-playbook -i '${self.public_ip},' \
        --private-key ~/.ssh/id_ed25519 \
        -u ubuntu \
        playbooks/deploy.yml
    EOT
  }
}
```

CI/CD Pipeline Integration (GitLab CI/CD)

This pipeline definition runs linting checks, builds a secure Execution Environment, and deploys configuration updates to production.

```
stages:
  - lint
  - build-ee
  - deploy

# 1. Lint the playbook to ensure best practices
lint:
  stage: lint
  image: python:3.10-slim
  script:
    - pip install ansible-lint
    - ansible-lint playbooks/deploy.yml

# 2. Build the execution environment container image
build_ee:
  stage: build-ee
  image: docker:24.0.7
  services:
    - docker:24.0.7-dind
  variables:
    DOCKER_HOST: tcp://docker:2375
  script:
```

```

- pip3 install ansible-builder
- ansible-builder build --tag $CI_REGISTRY_IMAGE/ee:latest
- docker login -u $CI_REGISTRY_USER -p $CI_REGISTRY_PASSWORD $CI_REGISTRY
- docker push $CI_REGISTRY_IMAGE/ee:latest

# 3. Execute the deployment inside the custom EE container
deploy_prod:
  stage: deploy
  image: $CI_REGISTRY_IMAGE/ee:latest
  script:
    - mkdir -p ~/.ssh && chmod 700 ~/.ssh
    - echo "$SSH_PRIVATE_KEY" > ~/.ssh/id_ed25519 && chmod 600 ~/.ssh/id_ed25519
    - ansible-playbook -i inventories/prod/site.ini playbooks/deploy.yml \
      --vault-password-file .vault_pass \
      --forks 50
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'

```

16. Comparison with Competing Tools

| Feature | Ansible | SaltStack | Terraform | Chef |

| :--- | :--- | :--- | :--- | :--- |

| Architecture | Agentless (Push over SSH/WinRM) | Agent-based or Agentless (ZeroMQ/SSH) | Agentless (API-driven) | Agent-based (Pull from Chef Server) |

| Primary Use Case | Configuration Management & App Deployment | High-speed Event-driven Orchestration | Infrastructure as Code (IaC) | Enterprise Configuration Management |

| State Management | Procedural & Idempotent (YAML) | Declarative/Procedural (YAML/Python) | Declarative State File (HCL) | Imperative (Ruby DSL) |

| Execution Latency | Low to Medium (Optimized with Pipelining) | Extremely Low (Sub-second via persistent agents) | Low (Direct API interactions) | Medium (Agent check-in intervals) |

| Cost / Licensing | Open Source / Red Hat AAP (Commercial) | Open Source / VMware Aria (Commercial) | Open Source (OpenTofu) / HashiCorp BSL | Open Source / Progress Chef (Commercial) |

| Scale Boundaries | ~5,000 nodes per controller (Scales higher with Automation Mesh) | 10,000+ nodes per Master node | Limited by Cloud Provider API rate limits | 10,000+ nodes per Chef Server |

17. Visual Cheat Sheet

Essential Configuration & Execution Parameters

```
????????????????????????????????????????????????????????????????????????????????????
?                                ANSIBLE PERFORMANCE CHEAT SHEET                                ?
????????????????????????????????????????????????????????????????????????????????????
?  ENVIRONMENT VARIABLES:                                                                ?
?    $ export ANSIBLE_PIPELINING=True          # Enable in-memory execution          ?
?    $ export ANSIBLE_FORKS=100                # Set high parallel workers           ?
?    $ export ANSIBLE_HOST_KEY_CHECKING=T      # Enforce strict host checking        ?
????????????????????????????????????????????????????????????????????????????????????
?  CLI COMMANDS:                                                                        ?
?    - Run dry-run with diff output:                                                    ?
?      $ ansible-playbook site.yml --check --diff                                     ?
?    - Encrypt sensitive strings:                                                       ?
?      $ ansible-vault encrypt_string 'secret' --name 'my_var'                       ?
?    - Run execution inside container:                                                  ?
?      $ ansible-navigator run site.yml --eei my-ee:latest                           ?
????????????????????????????????????????????????????????????????????????????????????
?  CRITICAL ANSIBLE.CFG SETTINGS:                                                       ?
?    [defaults]                                                                        ?
?    forks = 50                               # Parallel execution limit             ?
?    fact_caching = redis                     # Offload facts memory                 ?
?    [ssh_connection]                                                                    ?
?    pipelining = True                        # Skip disk writes on targets          ?
?    ssh_args = -o ControlPersist=30m         # Reuse active SSH connections        ?
????????????????????????????????????????????????????????????????????????????????????
```

18. Comprehensive Final Learning Summary

Key Takeaways

1. Optimize Executions with Pipelining and Sockets: In production, always enable `pipelining = True` and configure `ControlPersist` in your SSH settings. These options execute tasks directly in memory and reuse persistent network connections, reducing execution times by up to 90%.
2. Isolate Environments with Containers: Use `ansible-builder` and `ansible-navigator` to package your execution engine, dependencies, and collections into standardized Execution Environments. This guarantees consistent runs across developer machines, CI/CD pipelines, and production servers.
3. Secure Your Credentials: Never store plain-text secrets in code. Use Ansible Vault to encrypt individual variables, or fetch credentials dynamically from an external secrets manager like HashiCorp Vault during execution.
4. Scale with Automation Mesh: When managing large, multi-region infrastructures, deploy Ansible Automation Mesh. This architecture uses Hop and Execution nodes to securely route and run jobs across network

boundaries and firewalls without overloading a single control node.

Production Readiness Checklist

- [] ``pipelining`` is enabled in ``ansible.cfg``.
- [] ``ControlMaster`` and ``ControlPersist`` are configured to reuse SSH connections.
- [] ``host_key_checking`` is set to ``True`` to prevent Man-in-the-Middle attacks.
- [] All tasks handling sensitive data are marked with ``no_log: true``.
- [] Ansible Vault or an external KMS (like HashiCorp Vault) is used for secrets management.
- [] Playbooks are validated with ``ansible-lint`` in the CI/CD pipeline.
- [] The ``forks`` count is optimized to match the Control Node's memory and CPU capacity.

Q21. Ansible Execution Strategies (Linear vs Free vs Mitogen vs Pipelining)

Detailed Answer:

Ansible's default execution engine uses the ``linear`` strategy, where each task in a play is executed on all target hosts in parallel (up to the configured ``forks`` limit) before moving to the next task. While highly predictable, this creates a bottleneck if a single slow host delays the entire batch.

To optimize performance, Ansible provides alternative execution strategies:

1. Free Strategy (``strategy: free``): Allows each host to run tasks as fast as it can, independent of other hosts. It breaks step-synchronization, meaning Host A could be on Task 5 while Host B is still on Task 2. This is ideal for independent provisioning but unsuitable for multi-tier deployments requiring strict orchestration (e.g., database migration before application deployment).
2. Pipelining (``pipelining = True`` in ``ansible.cfg``): By default, Ansible copies module files to the remote target's temporary directory and executes them via a new SSH connection. Pipelining executes modules by piping the Python code directly into the remote Python interpreter's stdin without writing to the disk. This drastically reduces the number of SSH operations required per task from ~3-4 down to 1, cutting execution times by 50% to 80%.
Prerequisite: ``requiretty`` must be disabled in ``/etc/sudoers`` on target hosts.
3. Mitogen for Ansible: A third-party execution strategy that replaces the Ansible connection and execution engine entirely. Mitogen uses a custom multiplexed SSH protocol, persistent multiplexed connections, and intelligent caching of Python bytecode. It eliminates the overhead of spawning Python interpreters repeatedly, reducing CPU usage on both the control node and target hosts, and speeding up runs by up to 10x.

Production Scenario / Practical Example:

In a large-scale SRE runbook deploying an agent across 1,500 bare-metal servers, using the default ``linear`` strategy took 42 minutes. By tuning ``ansible.cfg`` to enable pipelining, increasing forks, and adopting the ``free`` strategy for independent agent installations, execution time was reduced to 4 minutes and 12 seconds.

``ansible.cfg``:

```
[defaults]
forks = 100
strategy = free
transport = ssh

[ssh_connection]
ssh_args = -o ControlMaster=auto -o ControlPersist=60s -o PreferredAuthentications=publickey
pipelining = True
```

Playbook Implementation:

```
---
- name: High-Performance Agent Deployment
  hosts: all
  strategy: free
  gather_facts: false # Skip fact gathering to save SSH round-trips
  tasks:
    - name: Copy agent package
      ansible.builtin.copy:
        src: /opt/software/monitoring-agent.rpm
        dest: /tmp/monitoring-agent.rpm
        mode: '0644'

    - name: Install agent
      ansible.builtin.dnf:
        name: /tmp/monitoring-agent.rpm
        state: present
        disable_gpg_check: true

---
```

Q22. Execution Environments (EEs) in Ansible Automation Platform (AAP)

Detailed Answer:

In Ansible Automation Platform (AAP) 2.x, the legacy concept of Virtual Environments (`virtualenvs`) was deprecated and replaced by Execution Environments (EEs). EEs are standardized, containerized environments (typically hosted on Podman or Docker) that package the Ansible runner, Ansible Core, Python dependencies, system libraries, and specific Ansible Collections.

This shift solves the "works on my machine" problem and prevents dependency drift across clustered control nodes. EEs are defined and built using `ansible-builder`, a utility that compiles user-defined requirements into a standard container image (e.g., based on Red Hat Universal Base Image - UBI).

An Execution Environment is defined by a `execution-environment.yml` file containing four primary sections:

1. version: The schema version.
2. build_arg_defaults: Base image definitions (e.g., `ee-supported-rhel8`).

3. dependencies:

- ``galaxy``: Path to a ``requirements.yml`` file for Ansible Collections.
- ``python``: Path to a ``requirements.txt`` file for Python libraries.
- ``system``: Path to a ``bindep.txt`` file for system-level dependencies (RPMs).

4. `additional_build_steps`: Custom RUN commands executed during image compilation.

Production Scenario / Practical Example:

To build an execution environment capable of managing AWS resources and interacting with HashiCorp Vault via specialized Python libraries, we configure the following ``execution-environment.yml`` and compile it.

``execution-environment.yml``:

```
version: 3
images:
  base_image:
    name: registry.redhat.io/ansible-automation-platform-24/ee-supported-rhel8:latest

dependencies:
  galaxy:
    collections:
      - name: amazon.aws
      - name: community.hashicorp
  python:
    - boto3
    - botocore
    - hvac>=2.1.0
  system:
    - xmlsec1-devel [platform:rpm]
    - gcc [platform:rpm]

additional_build_steps:
  prepend_base:
    - RUN dnf clean all && dnf update -y
  append_final:
    - RUN echo "EE built successfully on $(date)" > /etc/ee-build-info
```

Compilation Command:

```
ansible-builder build --tag private-registry.corp/infra/ee-aws-vault:v1.0.0
--container-runtime podman
```

Once built, this image is pushed to a private Automation Hub or container registry, and configured as the default Execution Environment inside the AAP Controller templates.

Q23. Ansible Vault Security Architecture & Multiple Vault Passwords

Detailed Answer:

Ansible Vault uses AES-256 (Advanced Encryption Standard) in CTR (Counter) mode with a SHA256 HMAC (Hash-based Message Authentication Code) to secure files, variables, and blocks of data. The encryption key is derived from a user-provided passphrase using PBKDF2 (Password-Based Key Derivation Function 2).

In complex multi-tenant or multi-environment architectures, a single vault password is a major security risk. It forces all teams or environments to share the same secret key. Ansible resolves this through Vault IDs (`--vault-id`). A Vault ID associates a label (e.g., ``dev``, ``prod``, ``database``) with a password source (a file, a script, or a prompt). This allows Ansible to selectively decrypt variables depending on the context, and encrypt different secrets with different keys within the same repository.

Security best practices dictate that vault passwords should never be stored in plaintext on disk. Instead, the vault-id should point to an executable script (a "vault client") that dynamically fetches the secret from an enterprise secrets manager (like HashiCorp Vault, AWS Secrets Manager, or CyberArk) using the local runner's IAM role or machine identity.

Production Scenario / Practical Example:

An enterprise has distinct secrets for ``staging`` and ``production``. We configure Ansible to dynamically resolve these keys by querying AWS Secrets Manager, ensuring no static keys are stored on the runner.

``ansible.cfg``:

```
[defaults]
vault_identity_list = staging@bin/get-staging-key.sh, production@bin/get-prod-key.sh
```

Dynamic Vault Key Client (``bin/get-prod-key.sh``):

```
#!/usr/bin/env bash
set -eo pipefail
# Fetch vault passphrase securely from AWS Secrets Manager using IAM role of the runner
aws secretsmanager get-secret-value \
  --secret-id "infra/ansible/prod-vault-key" \
  --query "SecretString" \
  --output text
```

*Ensure execution permissions: ``chmod +x bin/get-prod-key.sh``

Encrypting Variables with Specific Vault IDs:

```
# Encrypt staging secret
ansible-vault encrypt_string --vault-id staging 'StagingDBPassword123' --name 'db_password'

# Encrypt production secret
ansible-vault encrypt_string --vault-id production 'ProdDBPasswordSuperSecure!!!' --name
```

```
'db_password'
```

Resulting Playbook Variables File (`group\_vars/all.yml`):

```
---
# Staging and Prod secrets live side-by-side; Ansible decrypts only the one matched by the
active vault-id
staging_db_pass: !vault |
    $ANSIBLE_VAULT;1.2;AES256;staging
    313639313539656461623963623233303630653633333333373735393739666663737363635313837
    ...
prod_db_pass: !vault |
    $ANSIBLE_VAULT;1.2;AES256;production
    31313536333835616664653531393166303038626639353232656133373462376263303837393439
    ...
---
```

Q24. Inventory Plugins & Dynamic Inventory at Scale

Detailed Answer:

Legacy dynamic inventory scripts (executable Python files returning JSON) are deprecated in favor of Inventory Plugins. Inventory Plugins are native Python classes that hook directly into Ansible's core engine. They support configuration files (YAML), validation, built-in caching (using Redis, Memcached, or JSON files), and construct-based variable assignment.

When managing tens of thousands of instances across cloud providers (AWS, Azure, GCP), querying APIs on every playbook run is highly inefficient and risks API rate-limiting/throttling. To scale dynamic inventories, SREs must implement:

1. Fact and Inventory Caching: Storing retrieved metadata in a high-performance cache with a configured TTL.
2. Constructed Features: Grouping hosts dynamically inside the plugin using Jinja2 expressions, which offloads processing from the playbooks to the inventory loading phase.
3. Filtering and Pagination: Restricting API queries at the source (e.g., filtering by specific VPCs, resource groups, or tags) instead of fetching all objects and filtering downstream.

Production Scenario / Practical Example:

We configure a production-grade AWS EC2 inventory plugin (`aws\_ec2.yaml`) that queries instances across multiple regions, groups them dynamically by tags, caches results in Redis to avoid AWS API rate limits, and filters out non-running instances.

`production\_aws\_ec2.yaml`:

```
---
```

```
plugin: amazon.aws.aws_ec2
regions:
  - us-east-1
  - us-west-2
filters:
  # Reduce API payload by filtering at source
  instance-state-name: [running]
  tag:Environment: production

strict: false

# Construct dynamic groups based on tags
keyed_groups:
  - key: tags.Role
    prefix: role
  - key: placement.region
    prefix: region
  - key: tags.Application
    prefix: app

# Set variables dynamically
compose:
  ansible_host: private_ip_address
  ansible_user: 'ec2-user'

# Configure high-performance caching
use_extra_vars: true
cache: true
cache_plugin: ansible.builtin.redis
cache_connection: redis://localhost:6379/0
cache_timeout: 3600 # Cache for 1 hour
cache_prefix: aws_inventory_cache
```

Executing with Cached Inventory:

```
# Ensure the Redis server is running, then execute:
ansible-playbook -i production_aws_ec2.yaml deploy_app.yml
```

Q25. Ansible Pull Mode (`ansible-pull`)

Detailed Answer:

Standard Ansible operates in push mode (agentless, orchestrating over SSH/WinRM from a control node). While convenient, this model introduces scalability boundaries:

- **Network Bottlenecks**: The control node must maintain concurrent SSH channels, saturating bandwidth

and CPU when managing >5,000 hosts.

- **Firewall Restrictions**: Target environments must allow incoming SSH connections from the control node, which violates strict zero-trust network architectures.

`ansible-pull` reverses this model to a pull-based architecture (agent-like). It is a utility installed on each target node that runs via cron or a systemd timer. It:

1. Checkouts/pulls the latest configuration playbooks from a central Git repository.
2. Executes `ansible-playbook` locally over the `local` connection plugin (bypassing SSH entirely).
3. Applies configurations to itself.

This scales infinitely because the control node bottleneck is eliminated; the only central point of failure is the Git repository (which can be scaled via CDNs or read-replicas).

Production Scenario / Practical Example:

We configure a bootstrapping systemd service and timer on a fleet of 10,000 edge nodes to run `ansible-pull` every 15 minutes, pulling configurations from a secure GitLab repository.

Systemd Service File (`/etc/systemd/system/ansible-pull.service`):

```
[Unit]
Description=Ansible Pull Service
After=network-online.target
Wants=network-online.target

[Service]
Type=oneShot
ExecStart=/usr/bin/ansible-pull \
  -U https://gitlab-ci-token:${GITLAB_TOKEN}@gitlab.corp.internal/infra/edge-configs.git \
  -C main \
  -d /var/lib/ansible/pull \
  -i localhost, \
  local-provision.yml
User=root
StandardOutput=journal
StandardError=journal
```

Systemd Timer File (`/etc/systemd/system/ansible-pull.timer`):

```
[Unit]
Description=Run ansible-pull every 15 minutes

[Timer]
OnBootSec=2min
OnUnitActiveSec=15min
RandomizedDelaySec=120 # Prevent "thundering herd" on Git server
```

```
[Install]
WantedBy=timers.target
```

Command to bootstrap the node:

```
export GITLAB_TOKEN="glpat-SecureTokenExample"
envsubst < ansible-pull.service > /etc/systemd/system/ansible-pull.service
systemctl daemon-reload
systemctl enable --now ansible-pull.timer
```

Q26. Custom Ansible Modules in Python

Detailed Answer:

When Ansible's built-in modules are insufficient or executing complex shell commands becomes fragile, writing a custom Ansible module in Python is the professional solution. Custom modules must adhere to Ansible's design philosophies:

1. Idempotency: The module must check the current state against the desired state. If the target is already in the desired state, it must return `changed=False` without making modifications.
2. Structured JSON Output: Communication between Ansible and the module occurs via standard output using JSON.
3. Robust Error Handling: The module must catch exceptions, clean up temporary resources, and return structured error messages via `fail\_json()`.

Custom modules utilize the `AnsibleModule` utility class from `ansible.module\_utils.basic`. This class abstracts input argument parsing, validation (types, choices, mutual exclusion, requirements), and JSON output formatting.

Production Scenario / Practical Example:

We will build a custom module named `systemd\_override` that manages systemd service drop-in configuration overrides. It ensures a configuration directory exists, writes the override file, reloads systemd if changes were made, and remains fully idempotent.

Module Implementation (`library/systemd\_override.py`):

```
#!/usr/bin/python
# -*- coding: utf-8 -*-

from ansible.module_utils.basic import AnsibleModule
import os

def run_module():
```

```
module_args = dict(
    service=dict(type='str', required=True),
    override_name=dict(type='str', required=False, default='99-custom.conf'),
    content=dict(type='str', required=True, no_log=False),
    state=dict(type='str', choices=['present', 'absent'], default='present')
)

result = dict(
    changed=False,
    original_message='',
    message=''
)

module = AnsibleModule(
    argument_spec=module_args,
    supports_check_mode=True
)

service = module.params['service']
override_name = module.params['override_name']
content = module.params['content']
state = module.params['state']

target_dir = f"/etc/systemd/system/{service}.service.d"
target_file = f"{target_dir}/{override_name}"

# Check state
file_exists = os.path.exists(target_file)
current_content = ""
if file_exists:
    with open(target_file, 'r') as f:
        current_content = f.read()

# Determine if change is needed
change_needed = False
if state == 'present':
    if not file_exists or current_content.strip() != content.strip():
        change_needed = True
elif state == 'absent':
    if file_exists:
        change_needed = True

if module.check_mode:
    module.exit_json(changed=change_needed, **result)

if change_needed:
    if state == 'present':
        try:
```

```

        if not os.path.exists(target_dir):
            os.makedirs(target_dir, mode=0o755)
        with open(target_file, 'w') as f:
            f.write(content.strip() + "\n")
        os.chmod(target_file, 0o644)
        # Reload systemd daemon
        module.run_command("systemctl daemon-reload")
        result['message'] = f"Override {override_name} created and systemd reloaded."
    except Exception as e:
        module.fail_json(msg=f"Failed to create override: {str(e)}")
elif state == 'absent':
    try:
        os.remove(target_file)
        if not os.listdir(target_dir):
            os.rmdir(target_dir)
        module.run_command("systemctl daemon-reload")
        result['message'] = f"Override {override_name} removed."
    except Exception as e:
        module.fail_json(msg=f"Failed to remove override: {str(e)}")

    result['changed'] = True
else:
    result['message'] = "Systemd override is already in the desired state."

module.exit_json(**result)

if __name__ == '__main__':
    run_module()

```

Playbook Usage:

```

---
- name: Apply Systemd Custom Resource Limits
  hosts: app_servers
  tasks:
    - name: Configure custom memory limits for nginx
      systemd_override:
        service: nginx
        override_name: 10-limits.conf
        content: |
          [Service]
          MemoryMax=2G
          TasksMax=1000
        state: present

```

Q27. Custom Ansible Filters and Plugins

Detailed Answer:

Ansible leverages Jinja2 for templating. While Jinja2 provides standard filters (e.g., ``map``, ``select``), SREs often need custom domain-specific logic (e.g., parsing proprietary configuration strings, CIDR math, or cryptographic signatures). Writing custom Filter Plugins allows you to inject pure Python logic into Jinja2 templates.

Beyond Filter Plugins, Ansible supports other highly useful plugin types:

- **Lookup Plugins**: Run on the control node to fetch data from external sources (e.g., databases, key-value stores) and return it as a list of values inside tasks.
- **Callback Plugins**: Intercept Ansible execution events (e.g., task start, task failure, playbook stats) to redirect logs to platforms like Splunk, Datadog, or Elasticsearch.

Production Scenario / Practical Example:

We will implement:

1. A Filter Plugin that decrypts a custom ROT13/Base64 obfuscated string for legacy systems.
2. A Lookup Plugin that reads configuration directly from a local Consul instance.

Filter Plugin (``filter_plugins/security_filters.py``):

```
import codecs
import base64

def decode_obfuscated(value):
    """Decrypts a base64-encoded ROT13 string."""
    try:
        raw_bytes = base64.b64decode(value).decode('utf-8')
        return codecs.decode(raw_bytes, 'rot_13')
    except Exception as e:
        raise ValueError(f"Failed to decode string: {str(e)}")

class FilterModule(object):
    def filters(self):
        return {
            'decode_obfuscated': decode_obfuscated
        }
```

Lookup Plugin (``lookup_plugins/consul_kv.py``):

```
from ansible.plugins.lookup import LookupBase
from ansible.errors import AnsibleError
import urllib.request
import json

class LookupModule(LookupBase):
    def run(self, terms, variables=None, **kwargs):
        ret = []
```

```
consul_url = kwargs.get('consul_url', 'http://localhost:8500')

for term in terms:
    url = f"{consul_url}/v1/kv/{term}?raw=true"
    try:
        response = urllib.request.urlopen(url, timeout=5)
        ret.append(response.read().decode('utf-8').strip())
    except Exception as e:
        raise AnsibleError(f"Error querying Consul KV for key {term}: {str(e)}")
return ret
```

Using both in a Playbook:

```
---
- name: Demonstrate Custom Plugins
  hosts: localhost
  vars:
    # Obfuscated string: "SRE-Secret-Pass" base64 + rot13
    secure_payload: "RlVGLkZycXJndS5DbmZm"
  tasks:
    - name: Decrypt payload using custom filter
      ansible.builtin.debug:
        msg: "Decrypted: {{ secure_payload | decode_obfuscated }}"

    - name: Fetch DB Host from Consul using custom lookup
      ansible.builtin.debug:
        msg: "DB Host is {{ lookup('consul_kv', 'infrastructure/database/host',
consul_url='http://consul.corp.internal:8500') }}"
---
```

Q28. Optimizing Ansible Memory and CPU Consumption

Detailed Answer:

When running Ansible against thousands of nodes, the control machine often runs out of memory (OOM) or saturates its CPU. This is because Ansible forks a new process for every host connection. If `forks` is set to `500`, the control node may spin up 500 Python processes simultaneously.

To scale Ansible control nodes without crashing, SREs must tune execution parameters:

1. Reduce Fact Gathering Overhead: Fact gathering is highly resource-intensive. It runs multiple discovery commands on target hosts and transfers large JSON payloads back to the control node.

- Use `gather_facts: false` unless explicitly required.
- Implement **Fact Caching** with Redis or JSON files to persist facts across different playbook runs, specifying a high TTL.

2. Optimize Garbage Collection and Process Polling:

- `internal_poll_interval`: Configures how often Ansible checks internal queues. The default is `0.001` seconds (1ms). Raising this to `0.05` or `0.1` significantly reduces CPU utilization on the control node with virtually no impact on task execution speeds.

3. Limit SSH Multiplexing Memory Footprint:

- Ensure SSH `ControlMaster` is configured in `ansible.cfg` to reuse active connections instead of renegotiating TLS/TCP handshakes, reducing CPU overhead.

Production Scenario / Practical Example:

An enterprise Ansible controller with 16 vCPUs and 32GB RAM regularly crashed with OOM errors when executing runs across 2,000 servers. We applied the following performance-tuning configuration to stabilize the host and lower CPU usage by 65%.

Optimized `/etc/ansible/ansible.cfg`:

```
[defaults]
# Balance parallel execution with CPU capacity (typical formula: 2-4 x Cores)
forks = 64

# Reduce loop-checking frequency to save CPU cycles on control node
internal_poll_interval = 0.05

# Enable Redis Fact Caching to avoid querying facts on every play
gathering = smart
fact_caching = redis
fact_caching_timeout = 86400
fact_caching_connection = redis://127.0.0.1:6379/1

[ssh_connection]
# Optimize SSH multiplexing connection reuse
ssh_args = -o ControlMaster=auto -o ControlPersist=1800s -o
PreferredAuthentications=publickey
pipelining = True
control_path = %(directory)s/ansible-ssh-%%h-%%p-%%r
```

Q29. Managing Ansible State and Idempotency in Complex Orchestration

Detailed Answer:

Idempotency is the cornerstone of Infrastructure as Code (IaC). However, standard shell execution modules (`command`, `shell`, `raw`) are inherently non-idempotent because they run blindly on every execution. To maintain strict state management, SREs must instruct Ansible on how to determine state changes and failures.

To implement idempotency in custom orchestrations, use:

- `creates` / `removes`: Prevents task execution if a specified file already exists (or does not exist).
- `changed_when`: Overrides Ansible's default heuristic for what constitutes a "change". For example, if a script returns a specific exit code or stdout string when no changes are made, we map that condition to `changed: false`.
- `failed_when`: Overrides the default non-zero exit code failure mechanism. Useful for commands that return warnings as non-zero codes, or when an error string in `stdout` indicates a critical failure regardless of exit code.

Production Scenario / Practical Example:

We need to execute a legacy CLI tool (`/opt/bin/app-config`) that configures an application. The tool returns exit code `0` for success (even if nothing changed), exit code `2` if a change was successfully applied, and exit code `5` if the configuration is already in the desired state. Any other code is a true failure.

Idempotent Task Design:

```
- name: Manage Legacy App Configuration State
  hosts: app_servers
  tasks:
    - name: Run legacy application configuration
      ansible.builtin.command:
        cmd: /opt/bin/app-config --set max_connections=500 --db-host="db.corp.internal"
        # Prevent execution if the state lockfile already indicates this version is applied
        register: app_config_result

      # Override default exit-code handling
      failed_when:
        - app_config_result.rc not in [0, 2, 5]
        - "'CRITICAL_ERROR' in app_config_result.stdout"

      # Explicitly define what constitutes a change
      changed_when: "app_config_result.rc == 2"

    - name: Log configuration changes
      ansible.builtin.syslog:
        facility: local0
        priority: info
        msg: "Application configuration updated successfully."
        when: app_config_result.changed
```

Q30. Advanced Privilege Escalation (Become)

Detailed Answer:

Ansible's Become framework abstracts privilege escalation. While `sudo` is the most common execution plugin, enterprise systems often enforce stricter security controls using alternative escalation mechanisms like `pbrun` (PowerBroker), `pfexec` (Solaris), `doas` (OpenBSD), or custom wrapper scripts.

To secure privilege escalation at scale:

1. Never hardcode credentials: Use SSH keys and dynamic password lookup.
2. Unprivileged Become Users: When escalating to a non-root user (e.g., `become_user: postgres`), Ansible must copy temporary files to a path readable by that user. By default, Ansible uses POSIX ACLs. If the underlying filesystem does not support ACLs (e.g., NFS mounts, old filesystems), Ansible falls back to unprivileged file transfers which can leak secrets in `/tmp`. SREs must configure `allow_world_readable_tmpfiles = True` in `ansible.cfg` only as a last resort, or use pipelining to execute code directly in memory without writing files to disk.
3. Custom Become Flags: Provide custom arguments to the escalation command (e.g., preserving environment variables using `sudo -E`).

Production Scenario / Practical Example:

In a secure financial environment, root access is blocked. Operations must run through PowerBroker (`pbrun`) with environment variables preserved.

`ansible.cfg` Configuration:

```
[defaults]
# Force pipelining to avoid writing plaintext scripts to /tmp
pipelining = True
allow_world_readable_tmpfiles = False

[privilege_escalation]
become = True
become_method = community.general.pbrun
become_user = app_admin
become_flags = -u app_admin --profile custom_profile
```

Playbook Implementation:

```
---
- name: Secure Enterprise Privileged Playbook
  hosts: database_servers
  gather_facts: true
  tasks:
    - name: Stop Database Service
      ansible.builtin.systemd:
        name: postgresql
        state: stopped
      # Overriding escalations for a specific task if needed
      become: true
```

```
become_method: sudo
become_user: postgres
```

Q31. Ansible Collections Dependency Resolution & Private Automation Hub

Detailed Answer:

Ansible Collections isolate content (roles, modules, plugins) from the core engine. Managing dependencies across hundreds of production playbooks requires a deterministic packaging and distribution strategy.

Relying on public Ansible Galaxy for production deployments introduces risks:

- **Upstream Outages**: Builds fail if Galaxy is offline.
- **Supply Chain Attacks**: Malicious updates can be injected into public namespaces.
- **Lack of Air-Gapping**: Enterprise runners often have no direct internet access.

To mitigate this, enterprises deploy Private Automation Hubs (or JFrog Artifactory with Ansible repositories). We define dependencies in a `requirements.yml` file, specifying precise versions and signatures, and route resolution through the internal registry.

Production Scenario / Practical Example:

We configure an enterprise runner to resolve collections exclusively from an internal Private Automation Hub, enforcing signature verification for security.

`ansible.cfg`:

```
[galaxy]
# Direct all dependency resolution to the Private Automation Hub
server_list = private_hub

[galaxy_server.private_hub]
url = https://hub.corp.internal/api/galaxy/
username = automation-runner
password = '{{ lookup("env", "HUB_TOKEN") }}'
# Enforce GPG signature checking for downloaded collections
signature_verification_keyring = /etc/ansible/keyring.gpg
```

`requirements.yml`:

```
---
collections:
  - name: amazon.aws
    version: ">=6.1.0"
    source: private_hub
```

```
- name: kubernetes.core
  version: ">=2.4.0,<3.0.0"
  source: private_hub
```

Installation Command via CI/CD Pipeline:

```
# Verify GPG signatures and download dependencies to local project directory
ansible-galaxy collection install -r requirements.yml -p ./collections --verify
```

Q32. Asynchronous Task Execution and Polling

Detailed Answer:

By default, Ansible holds the SSH connection open for the entire duration of a task. If a task takes an hour to run (e.g., a database backup, OS upgrade, or disk formatting), the SSH connection may drop due to network timeouts, firewalls, or packet loss, causing the entire play to fail even if the remote task succeeded.

To prevent this, Ansible supports Asynchronous Execution:

- **``async``**: Tells Ansible to run the task in the background. The value specifies the maximum time (in seconds) the task is allowed to run before being killed.
- **``poll``**: Specifies how often (in seconds) Ansible should connect to the host to check if the task has completed.
- If `poll > 0`, the playbook blocks and waits, but queries the status periodically.
- If `poll: 0`, Ansible starts the task in the background and immediately moves to the next task without waiting. This is known as "fire-and-forget".

To track fire-and-forget tasks, save the task's registration metadata (which includes an `ansible_job_id`) and use the `ansible.builtin.async_status` module to query and block later in the playbook.

Production Scenario / Practical Example:

We need to trigger a large database schema migration that takes up to 45 minutes, run a quick independent system check in parallel, and then block until the migration completes.

Playbook Implementation:

```
---
- name: Long-Running Database Migration
  hosts: db_primary
  tasks:
    - name: Trigger DB Migration asynchronously (Fire-and-Forget)
      ansible.builtin.command:
        cmd: /opt/db/bin/migrate-schema.sh --env production
      # Allow up to 1 hour (3600 seconds), do not poll, move to next task immediately
```

```
    async: 3600
    poll: 0
    register: db_migration_job

- name: Perform Independent Local System Checks
  ansible.builtin.service_facts:
  delegate_to: localhost

- name: Monitor DB Migration Status and Wait for Completion
  ansible.builtin.async_status:
    jid: "{{ db_migration_job.ansible_job_id }}"
  register: job_result
  # Poll every 30 seconds until completed or 3600 seconds elapsed
  until: job_result.finished
  retries: 120
  delay: 30
  failed_when: "'ERROR' in job_result.stderr or job_result.rc != 0"
```

Q33. Designing High-Availability Ansible Automation Platform (AAP) Architecture

Detailed Answer:

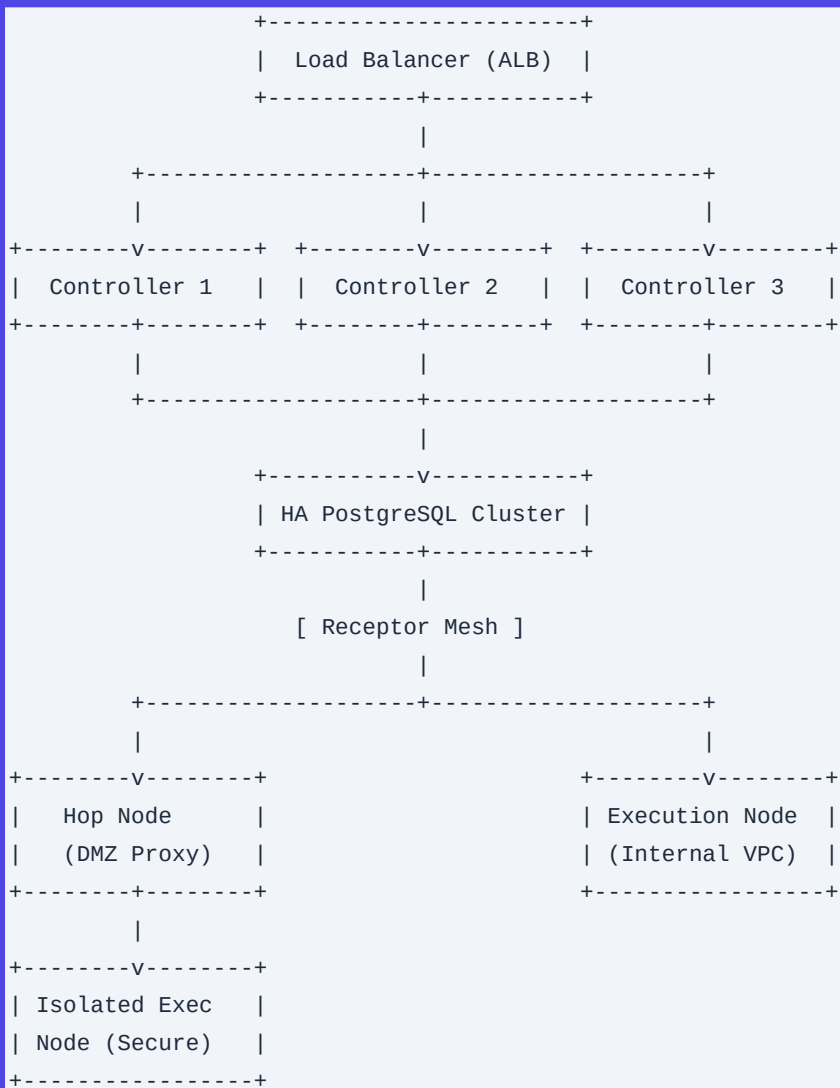
An enterprise-grade Ansible Automation Platform (AAP) 2.x architecture separates the management plane from the execution plane using a mesh network (Receptor).

Core Architectural Components:

1. Controller Nodes (Web/API): Serve the user interface, API, and RBAC engine. They store configuration state in a highly available external PostgreSQL cluster.
2. Execution Nodes: Run the actual playbooks inside Execution Environments (containers).
3. Hop Nodes: Act as secure network proxies/routers. They forward traffic from the controller to execution nodes that reside in isolated, private network segments (e.g., DMZs or VPCs without direct controller access).
4. Receptor: The underlying peer-to-peer mesh networking service that routes control messages, payloads, and files across nodes.

Scalability & High Availability (HA) Design:

- Deploy a minimum of 3 Controller Nodes behind an Application Load Balancer (ALB) doing active health checks.
- Externalize the PostgreSQL database into an HA configuration (e.g., AWS Aurora or Patroni) with automatic failover.
- Group Execution Nodes into **Instance Groups**. If an execution node fails, the Controller automatically reschedules pending jobs to other healthy nodes in the group.



Production Scenario / Practical Example:

An infrastructure configuration (`inventory.ini`) used by the `ansible.builtin.setup` or installation playbooks to deploy a 3-node HA Controller, an external PostgreSQL cluster, and a mesh network containing a Hop Node and an Isolated Execution Node.

Production AAP Installer Inventory (`inventory`):

```

[automationcontroller]
controller-01.corp.internal node_type=control
controller-02.corp.internal node_type=control
controller-03.corp.internal node_type=control

[database]
postgres-primary.corp.internal db_backup_enabled=true
postgres-replica.corp.internal

```

```
[execution_nodes]
exec-01.corp.internal node_type=execution
exec-02.corp.internal node_type=execution

[hop_nodes]
dmz-hop.corp.internal node_type=hop

[isolated_nodes]
secure-exec.corp.internal node_type=execution peers=dmz-hop.corp.internal

[all:vars]
pg_host='postgres-primary.corp.internal'
pg_port='5432'
pg_database='awx'
pg_username='awx'
pg_password='ComplexDatabasePasswordHere'
pg_sslmode='verify-full'
```

Q34. Ansible Handlers Advanced Execution

Detailed Answer:

Handlers are special tasks that only run when "notified" by another task that has reported a state change (`changed=True`). While simple in theory, complex production deployments require deep control over when and how handlers execute:

1. Flush Handlers (`meta: flush_handlers`): By default, handlers run at the very end of the play. If a task in the middle of a play requires a service to be restarted before proceeding (e.g., starting an agent that depends on a newly written configuration file), you must force Ansible to run all queued handlers immediately using the `ansible.builtin.meta: flush_handlers` task.
2. Handler Chains: Handlers can notify other handlers. If Handler A restarts a service, and Handler B verifies the service health, Handler A can notify Handler B.
3. Listen Directive: Handlers can "listen" to generic topics. Multiple tasks can notify a single generic event name (e.g., `restart web servers`), and all handlers listening to that topic will execute. This decouples the tasks from specific handler names.

Production Scenario / Practical Example:

We deploy an Nginx configuration change, reload Nginx immediately to verify the configuration, and then run a health check. If the health check fails, we roll back the configuration.

Playbook Implementation:

```
---
- name: Zero-Downtime Nginx Configuration Rollback
  hosts: webserver
  force_handlers: true # Ensure handlers run even if subsequent tasks fail
  tasks:
    - name: Backup current Nginx config
      ansible.builtin.copy:
        src: /etc/nginx/nginx.conf
        dest: /etc/nginx/nginx.conf.bak
        remote_src: true

    - name: Write new Nginx configuration
      ansible.builtin.template:
        src: nginx.conf.j2
        dest: /etc/nginx/nginx.conf
        mode: '0644'
      notify: Reload Nginx

    # Force the "Reload Nginx" handler to execute NOW, not at the end of the play
    - name: Apply configuration changes immediately
      ansible.builtin.meta: flush_handlers

    - name: Verify application health locally
      ansible.builtin.uri_to_validate:
        url: http://localhost:80/health
        status_code: 200
      register: health_check
      ignore_errors: true

    - name: Rollback configuration on failure
      ansible.builtin.copy:
        src: /etc/nginx/nginx.conf.bak
        dest: /etc/nginx/nginx.conf
        remote_src: true
      notify: Reload Nginx
      when: health_check.failed

  handlers:
    - name: Reload Nginx
      ansible.builtin.systemd:
        name: nginx
        state: reloaded
        listen: "restart web services"
---
```

Q35. Securing Ansible: Task Isolation, Sandboxing, and No-Log

Detailed Answer:

Ansible runs on the control node with the permissions of the user executing it. If a playbook is poorly designed or processes untrusted input, it can leak sensitive credentials or compromise the control node itself. SREs must implement strict security controls:

1. ``no_log: true``: Prevents tasks from logging sensitive variable values (passwords, private keys, tokens) to standard output, system logs (syslog), or the AAP database. *Note: ``no_log`` does not prevent debugging if ``ansible.cfg`` has debugging flags enabled globally, but it secures standard production runs.*
2. Restricting Local Execution: Tasks using ``local_action`` or ``delegate_to: localhost`` execute commands directly on the Ansible control node. If these commands use untrusted variables (e.g., user input from a web portal), they are vulnerable to Command Injection. SREs should disable local action capabilities for non-admin users or execute playbooks inside isolated Execution Environments (containers) running as non-root users.
3. Environment Sandboxing: Ensure target configurations are validated using dry-run modes (``--check``) and assert statements before applying changes to production.

Production Scenario / Practical Example:

We implement a playbook that retrieves a temporary database token from HashiCorp Vault, writes it to a configuration file, and ensures that the sensitive token is never leaked in standard output or system logs.

Secure Playbook Implementation:

```
---
- name: Secure Database Credential Rotation
  hosts: app_servers
  vars:
    vault_addr: "https://vault.corp.internal:8200"
  tasks:
    - name: Retrieve dynamic DB credentials from Vault
      community.hashicorp.vault_read:
        url: "{{ vault_addr }}"
        path: "database/creds/app-role"
        auth_method: token
        token: "{{ lookup('env', 'VAULT_TOKEN') }}"
      register: db_secret
      delegate_to: localhost
      # CRITICAL: Prevent secrets from being logged to stdout/syslog
      no_log: true

    - name: Ensure configuration directory exists
      ansible.builtin.file:
        path: /etc/app
        state: directory
        mode: '0700'
        owner: app-user
```

```
- name: Write credentials to local config file
  ansible.builtin.template:
    src: db_creds.conf.j2
    dest: /etc/app/db_creds.conf
    mode: '0600'
    owner: app-user
  # CRITICAL: Prevent template rendering output from being logged
  no_log: true
  vars:
    db_username: "{{ db_secret.data.username }}"
    db_password: "{{ db_secret.data.password }}"
```

Q36. Rolling Updates and Zero-Downtime Deployments with Ansible

Detailed Answer:

In high-availability web architectures, deploying updates to all servers at once causes service downtime. Ansible provides native orchestration directives to manage rolling deployments and control execution flow:

1. ``serial``: Limits how many hosts Ansible manages at one time in a play. You can specify a fixed number (e.g., ``serial: 3``) or a percentage (e.g., ``serial: 25%``). Ansible will complete the *entire* play for that batch of hosts before starting the next batch.
2. ``max_fail_percentage``: Used in conjunction with ``serial``. It defines the maximum percentage of hosts in a batch that can fail before the entire playbook run is aborted. This acts as an automated circuit breaker, preventing a faulty configuration from rolling out to the entire cluster if the first batch fails.
3. ``throttle``: Limits the number of concurrent tasks on a specific task level, rather than the entire play level.

Production Scenario / Practical Example:

We deploy a rolling software update across a cluster of 100 web servers. We update them in batches of 10% (10 servers at a time). For each batch, we remove the hosts from the F5 Load Balancer, update the software, verify health, and re-add them to the load balancer. If more than 20% of a batch fails, we abort the run.

Playbook Implementation:

```
---
- name: Zero-Downtime Rolling App Deployment
  hosts: webservers
  serial: "10%"
  max_fail_percentage: 20
  tasks:
    - name: Remove node from F5 Load Balancer pool
      f5networks.f5_modules.bigip_pool_member:
```

```
state: forced-offline
pool: "web-pool"
partition: "Common"
host: "{{ ansible_host }}"
port: 80
provider:
  server: "f5.corp.internal"
  user: "admin"
  password: "{{ f5_password }}"
delegate_to: localhost

- name: Pull and Update Application Container
  containers.podman.podman_image:
    name: "registry.corp.internal/apps/web"
    tag: "v2.1.0"
    state: build

- name: Restart Application Container
  containers.podman.podman_container:
    name: web-app
    image: "registry.corp.internal/apps/web:v2.1.0"
    state: started
    restart: true

- name: Wait for application to pass local health check
  ansible.builtin.uri:
    url: "http://localhost:8080/health"
    status_code: 200
  register: local_health
  until: local_health.status == 200
  retries: 6
  delay: 10

- name: Add node back to F5 Load Balancer pool
  f5networks.f5_modules.bigip_pool_member:
    state: enabled
    pool: "web-pool"
    partition: "Common"
    host: "{{ ansible_host }}"
    port: 80
    provider:
      server: "f5.corp.internal"
      user: "admin"
      password: "{{ f5_password }}"
  delegate_to: localhost
```

Q37. Deep Dive: Variable Precedence and Scope in Complex Playbooks

Detailed Answer:

Ansible has 22 levels of variable precedence. Understanding this hierarchy is critical for designing complex, reusable roles and debugging variable shadowing issues.

Simplified Precedence Order (from lowest to highest):

1. Role Defaults (``roles/x/defaults/main.yml``): Intended to be easily overridden.
2. Inventory File or Script Group Vars: Variables defined directly in the inventory.
3. Inventory Group Vars (``group_vars/all`` then ``group_vars/webserver``): Variables in the project structure.
4. Inventory Host Vars (``host_vars/node1``).
5. Playbook Group/Host Vars: Defined within the playbook itself.
6. Host Facts / Cached Facts: Discovered facts (e.g., ``ansible_distribution``).
7. Registered Variables / Set Facts: Set during runtime using ``register`` or ``set_fact``.
8. Role Variables (``roles/x/vars/main.yml``): Intended to be constant; difficult to override.
9. Extra Vars (``-e`` or ``--extra-vars`` via CLI): Always wins.

To maintain clean architecture:

- Keep roles configurable by placing defaults in ``defaults/main.yml``.
- Avoid using ``vars/main.yml`` in roles unless the variables must never be overridden.
- Use ``set_fact`` when you need to dynamically calculate a variable during execution that should override inventory configurations.

Production Scenario / Practical Example:

We want to verify variable resolution and override behavior across roles, group variables, and dynamic facts.

Role Defaults (``roles/web/defaults/main.yml``):

```
---
app_port: 8080
```

Group Variables (``group_vars/production.yml``):

```
---
app_port: 9000
```

Playbook Execution:

```
---
- name: Variable Precedence Demonstration
  hosts: production_servers
```

```
roles:
  - web
tasks:
  - name: Debug port before set_fact
    ansible.builtin.debug:
      msg: "Active Port (should be 9000 from group_vars): {{ app_port }}"

  - name: Override port dynamically at runtime
    ansible.builtin.set_fact:
      app_port: 9999

  - name: Debug port after set_fact
    ansible.builtin.debug:
      msg: "Active Port (should be 9999 from set_fact): {{ app_port }}"
```

If this playbook is executed with:

```
ansible-playbook -i inventory playbook.yml -e "app_port=1111"
```

The active port will be `1111` in both debug statements because command-line Extra Vars override all other levels, including `set\_fact`.

Q38. Ansible Fact Caching and Custom Fact Modules

Detailed Answer:

By default, Ansible runs a massive discovery process (fact gathering) at the start of every play. For a fleet of 2,000 servers, this is a major performance bottleneck. SREs can optimize this by configuring Fact Caching and deploying Custom Local Facts.

Fact caching stores discovered metadata in an external backend (e.g., Redis, Memcached, or flat JSON files) so subsequent playbook runs can read the state instantly without querying the target hosts.

Furthermore, you can define Custom Facts on target hosts by placing executable scripts or static JSON/INI files in `/etc/ansible/facts.d/\*.fact`. During the gathering phase, Ansible reads these files and populates them under the `ansible\_local` namespace.

Production Scenario / Practical Example:

We deploy a custom Python script to 1,000 servers that queries the local hardware RAID controller status. This script exposes custom facts, which are then cached in a Redis server on the controller to speed up subsequent runs.

Custom Fact Script (`/etc/ansible/facts.d/raid.fact`):

```
#!/usr/bin/env python3
import json
import subprocess

def get_raid_status():
    try:
        # Mocking a RAID controller check command
        # In production: output = subprocess.check_output(["megacli", "-AdpAllInfo",
        "-aAll"])
        return {"status": "HEALTHY", "disks": 4, "type": "RAID-10"}
    except Exception:
        return {"status": "UNKNOWN", "disks": 0, "type": "NONE"}

if __name__ == "__main__":
    print(json.dumps(get_raid_status()))
```

Ensure execution permissions: `chmod +x /etc/ansible/facts.d/raid.fact`

`ansible.cfg` Configuration on the Controller:

```
[defaults]
gathering = smart
fact_caching = redis
fact_caching_connection = redis://redis.corp.internal:6379/2
fact_caching_timeout = 86400 # Cache for 24 hours
```

Playbook Utilizing the Custom Fact:

```
---
- name: Monitor Server RAID Health
  hosts: bare_metal
  tasks:
    - name: Fail play if RAID status is not healthy
      ansible.builtin.fail:
        msg: "RAID controller is reporting degraded state!"
      # Accessing the custom local fact
      when: ansible_local.raid.status != 'HEALTHY'
```

Q39. Real-time Event-Driven Ansible (EDA)

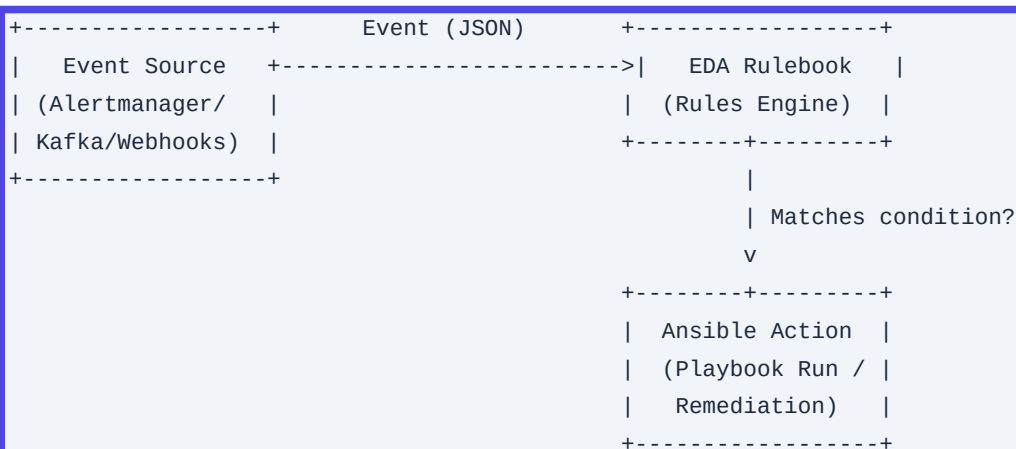
Detailed Answer:

Traditional Ansible is reactive and proactive (triggered manually, via schedules, or through CI/CD pipelines). Event-Driven Ansible (EDA) introduces a declarative, real-time automation layer that responds to system events (e.g., alerts, webhooks, log messages) without human intervention.

Core Components of EDA:

1. Sources: Input plugins that listen for events (e.g., Prometheus Alertmanager webhooks, Kafka topics, AWS SQS, or file watches).
2. Rules (Rulebooks): Declarative structures written in YAML that define conditions based on event payloads (e.g., "If Prometheus sends an alert with status firing and alertname high_cpu").
3. Actions: The automated response to the event, such as running a specific Ansible playbook, injecting facts, or calling an API.

This architecture enables auto-remediation (e.g., automatically restarting a failed service, scaling up a Kubernetes deployment, or collecting diagnostic logs during an outage).



Production Scenario / Practical Example:

We configure an Event-Driven Ansible rulebook to listen for Prometheus Alertmanager alerts. When a "High Memory Usage" alert is received, EDA automatically triggers a remediation playbook that safely clears application caches on the target host.

EDA Rulebook (`rulebooks/remediate\_memory.yml`):

```

---
- name: Listen for Prometheus Alerts
  hosts: all
  sources:
    - sscore.webhook:
        host: 0.0.0.0
        port: 5000
        endpoint: /alerts

  rules:
    - name: Remediate High Memory Alert
      condition: event.payload.status == "firing" and event.payload.commonLabels.alertname ==
"HighMemoryUsage"
  
```

```
action:
  run_playbook:
    name: playbooks/clear_cache.yml
    extra_vars:
      target_host: "{{ event.payload.commonLabels.instance }}"
```

Remediation Playbook (`playbooks/clear\_cache.yml`):

```
---
- name: Automated Cache Remediation
  hosts: "{{ target_host }}"
  gather_facts: false
  tasks:
    - name: Log remediation start to syslog
      ansible.builtin.syslog:
        facility: daemon
        priority: warning
        msg: "EDA triggered memory remediation for {{ inventory_hostname }}."

    - name: Clear application redis cache
      ansible.builtin.command:
        cmd: redis-cli flushall
      ignore_errors: true

    - name: Restart application service
      ansible.builtin.systemd:
        name: node-app
        state: restarted
```

Q40. Ansible Integration with HashiCorp Vault for Dynamic Secrets

Detailed Answer:

Hardcoding secrets in Git repositories-even if encrypted with Ansible Vault-creates lifecycle management challenges. If a secret is rotated in your enterprise vault, you must re-encrypt and commit the new value to your Git repository.

Integrating Ansible with HashiCorp Vault using lookup plugins resolves this. The control node dynamically fetches secrets from HashiCorp Vault at runtime, keeping secrets in memory and preventing them from ever being written to disk in plain text.

To set up enterprise-grade integration:

1. Authentication: Use HashiCorp Vault AppRole or JWT/OIDC (OpenID Connect) authentication. SRE runners authenticate to Vault using a secure role ID and secret ID (or CI/CD JWT tokens) to obtain a temporary client token.

2. Lookup Plugin: Use `community.hashicorp.hashi\_vault` to query secrets dynamically.

3. Task Isolation: Combine the lookup with `no\_log: true` to prevent secrets from being printed in the terminal output.

Production Scenario / Practical Example:

We configure a playbook that runs in a CI/CD pipeline. The pipeline injects a Vault AppRole Role ID and Secret ID. The playbook authenticates to Vault, retrieves a dynamic PostgreSQL database credential, and configures the application.

Playbook Implementation:

```
---
- name: Deploy App with Dynamic Vault Secrets
  hosts: app_servers
  vars:
    vault_url: "https://vault.corp.internal:8200"
  tasks:
    - name: Fetch Database Credentials from Vault
      ansible.builtin.set_fact:
        db_secrets: "{{ lookup('community.hashicorp.hashi_vault',
                                'secret=secret/data/production/database:data',
                                url=vault_url,
                                auth_method='approle',
                                role_id=lookup('env', 'VAULT_ROLE_ID'),
                                secret_id=lookup('env', 'VAULT_SECRET_ID')) }}"

      delegate_to: localhost
      no_log: true

    - name: Write application configuration file
      ansible.builtin.template:
        src: app_config.json.j2
        dest: /opt/app/config.json
        mode: '0600'
        owner: app-user
        group: app-user
      no_log: true
      vars:
        db_user: "{{ db_secrets.username }}"
        db_pass: "{{ db_secrets.password }}"
```

Jinja2 Template (`templates/app\_config.json.j2`):

```
{
  "database": {
    "host": "prod-db.corp.internal",
    "port": 5432,
```

```
"username": "{{ db_user }}",  
"password": "{{ db_pass }}"  
}  
}
```